Ralph Warrand

Software Engineer, Games & Simulation

Eindhoven, the Netherlands · ralphwarrand@gmail.com · giralfe.github.io/portfolio · github.com/giralfe · linkedin.com/in/ralphwarrand

I write software for games and for research: gameplay and AI systems, engines and renderers, physics solvers, and the tools around them. Four years making games at BUas, two of them released on Steam, taught me frame budgets, profiling and working with designers and artists. My MSc at TU/e, with a thesis on constrained multibody simulation co-supervised by Algoryx's Claude Lacoursière, taught me to derive the model first and check results against something outside my own code. Dutch and English native.

Education

MSc Computer Science & Engineering · Eindhoven University of Technology 2024 – March 2027 (expected)

Thesis: *An Entity Component System for Extensible 2D Constrained Multibody Simulation*, with dr. Andrei Jalba (TU/e) and dr. Claude Lacoursière (Umeå University / Algoryx). Pre-master 2024–25. Courses include Simulation in Computer Graphics, Stochastic Simulation, Geometric Algorithms.

BSc Creative Media & Game Technologies · Breda University of Applied Sciences 2020 – 2024

Programming track: C++, engines, rendering and game AI across sixteen team projects.

International Baccalaureate Diploma · International School Eindhoven · HL Maths, Physics, English 2018 – 2020

Games

S.P.L.I.C.E.D. · Fishbait Studios, BUas · UE5 · released free on Steam 2022 – 2023

AI and gameplay programmer and programming lead over a full academic year. Built the multi-phase boss with its tuning in Data Assets for the designer, enemy AI, 360-degree aiming and an editor collision tool. Ran code review for the team (100+ reviews in the final block) and migrated the project to UE 5.1.

Small Dragon Big Appetite · graduation project · UE5 · released free on Steam 2024

Sole programmer for the final quarter after a 48-hour jam. Profiled the enemy crowd with Unreal Insights and took the full level from 77 ms to 12.5 ms per frame with distance culling and LODs; prototyped the fireball four ways with a recruited playtester group; added ability unlocks.

Verdant Games · student studio, BUas incubator · UE5, C++ 2023 – 2024

Lead programmer on the core team for the first half of the year, then programmer; set up the code-review pipeline. Procedural dungeon generation from research document to a playable loop: room separation, Delaunay triangulation, minimum spanning tree, loop extraction, level streaming, a graph editor for designers.

SUB NIVIS · BUas team · custom C++ engine, PC and PlayStation 4 2022

Retro FPS on a team-built engine. Brought Quake's BSP and WAD formats in so designers could build levels in TrenchBroom, wrote the enemy AI, set up the PlayStation 4 build, took a 512-enemy stress test from 7.4 to 5.3 ms. Product owner for the release block.

Physics and research

- **Flatland: 2D multibody physics engine (MSc thesis)** (2026–, C++17 / Python). discrete-time Lagrangian mechanics: joints, ropes, motors and frictional contact as exact constraints, elasticity as compliant constraints, one Spook stepper and one boxed-LCP solve. I rebuilt the research library around an entity component system so bodies of any degree-of-freedom width feed one solver, in a C++17 core driven from Python. Measured: a pendulum holds its constraint to 1.6×10^{-5} m, energy stays within 0.25% over 20 000 steps, 250 disks step in 0.74 ms. A symbolic verifier re-derives the algebra and fails the build if theory and code disagree.
- **HexForge: GPU particle-physics engine** (2024–, C++20 / CUDA / OpenGL). fluids (Position Based Fluids), cloth (XPBD), sand and rigid bodies in one CUDA kernel suite, with screen-space fluid rendering. Spacing, kernel radius, substep bound and damping derived from physical inputs; each kernel traced to its paper.
- **Particletoys: particle, cloth and fluid simulation** (2026, C++, team of 3, TU/e). my parts: a constraint solver (Lagrange multipliers, Baumgarte, matrix-free conjugate gradient); a measured comparison of four integrators whose stable step matched a hand estimate from the cloth's stiffness; cloth; and on Stam's fluid solver a level-set free surface, two-way rigid-body coupling, buoyancy and temperature.
- **scrye: electron-microscopy simulation** (2026, Python / NumPy). Monte Carlo electron transport (backscatter, secondaries, characteristic X-rays, energy loss, knock-on damage) and a Cowley–Moodie multislice engine for HRTEM, STEM and diffraction.
- **MochaNES: NES emulator** (2025–, Java). CPU, PPU, APU and six mappers, measured against blargg's hardware test ROMs (35 of 51 pass), with a physically motivated CRT model and a browser build.

Skills

Languages	C++17/20, Python (NumPy, SciPy, SymPy), CUDA, Java, Rust, GLSL / HLSL / WGSL
Physics & numerics	Spook / boxed LCP, XPBD / PBF, Stable Fluids, implicit integration, Monte Carlo, Eigen
Games & graphics	Unreal Engine 5 (C++, Blueprints), behaviour trees, OpenGL, WebGPU, DX12, PS4
Tools	Git, Perforce, CMake, GitHub Actions, Jira, doctest / pytest / googletest, Unreal Insights, Optick